

# محاسبي — الخطة التقنية (Tech Plan)

الوثيقة دي هي دليل التنفيذ. الـ Features والـ Billing موجودين في `PLAN.md`، وهنا **إزاي هبنينهم**: الـ Stack، الهيكل، القواعد، الداتايز، الـ APIs، المزامنة الأوفلاين، الـ Real time، الاختبارات، النشر، وخطة أول أسابيع يوم بيوم.

## 0. القرارات الأساسية (ملخص)

| الموضوع                  | القرار                                                                                                                                 |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Backend                  | <b>Laravel</b> (آخر إصدار مستقر x/13.x.12) + <b>Modular Monolith: Modules</b> — <b>PHP 8.4</b> مستقلة <b>تتفعل لكل محل حسب اشتراكه</b> |
| التواصل بين الـ Modules  | <b>Event-Driven: Domain Events + Transactional Outbox + Listeners</b> في الـ <b>Queue</b> (مش Event Sourcing)                          |
| قاعدة البيانات           | <b>PostgreSQL 17</b>                                                                                                                   |
| Real time                | <b>Laravel Reverb</b> (WebSockets) + <b>Laravel Echo</b>                                                                               |
| Queues                   | <b>Redis + Laravel Horizon</b>                                                                                                         |
| Auth                     | <b>Sanctum</b> : كوكيز للوحة الويب، Tokens لأجهزة الـ POS وتطبيق المالك                                                                |
| الصلاحيات                | <b>spatie/laravel-permission</b> بوضع <b>Teams</b> (الـ Team = المحل)                                                                  |
| Frontend                 | <b>Nuxt 4</b> + <b>TypeScript</b> + <b>Nuxt UI (RTL)</b> + <b>Pinia</b> — تطبيق واحد فيه اللوحة والكاشير                               |
| الكاشير أوفلاين          | <b>PWA</b> + <b>IndexedDB (Dexie)</b> + <b>Outbox Sync</b>                                                                             |
| تطبيق المالك             | <b>Flutter</b> + <b>Riverpod</b> + <b>Dio</b> + <b>go_router</b>                                                                       |
| لوحة إدارة المنصة (إحنا) | <b>Filament</b> داخل نفس مشروع <b>Laravel</b>                                                                                          |
| توثيق الـ API            | <b>Scramble</b> (OpenAPI تلقائي) ← توليد <b>TypeScript types</b> للـ <b>Nuxt</b>                                                       |
| الفلوس                   | <b>integer بالقرش</b> + <b>brick/money</b>                                                                                             |
| IDs                      | <b>UUIDv7</b> (بيتولد على الجهاز في الأوفلاين ومترتب زمنياً)                                                                           |
| الاختبارات               | <b>Pest</b> + <b>Larastan</b> + <b>Pint / Vitest</b> + <b>Playwright</b>                                                               |
| النشر                    | <b>Cloudflare (DNS/R2)</b> + <b>Sentry</b> + <b>(Laravel Forge أو Coolify)</b> <b>Docker</b> + <b>VPS</b>                              |

**ليه Modular Monolith؟** مشروع واحد، Deploy واحد، داتايز واحدة — أسرع حاجة لمطور واحد. والتقسيم لـ Modules بحدود واضحة، والتواصل بينها بالـ Events، بيخلي أي Module يتفعل أو يتفعل لمحل معين من غير ما يآثر على الباقي، ويسهل فصل أي جزء كخدمة لوحده بعدين لو احتجنا.

## 1. هيكل الـ Repo (Monorepo)

```
muhasabi-egypt/
├── apps/
│   ├── api/           # Laravel - API + Reverb + Horizon + Filament (المنصة admin)
│   ├── web/           # Nuxt 4 - لوحة المحل + الكاشير (PWA)
│   └── owner/         # تطبيق المالك - Flutter
├── packages/
│   └── api-types/     # TypeScript types من OpenAPI (Scramble)
├── infra/
│   ├── docker/        # Dockerfiles + nginx + supervisor
│   └── compose.yml     # بيئة التطوير المحلية
├── docs/              # ADRs + ملاحظات تقنية
├── PLAN.md
├── TECH_PLAN.md
└── .github/workflows/ # CI
```

## 2. بيئة التطوير المحلية

فيه: `infra/compose.yml`

| Port | الاستخدام                                | Service             |
|------|------------------------------------------|---------------------|
| 5432 | الداتابيز                                | pgsql (postgres:17) |
| 6379 | Queue + Cache + Sessions                 | redis (redis:7)     |
| 8025 | استقبال الإيميلات في التطوير             | mailpit             |
| 9000 | S3 محلي (صور البطايق، الأجهزة، المرفقات) | minio               |
| 8000 | FrankenPHP أو php artisan serve          | api                 |
| 8080 | php artisan reverb:start                 | reverb              |
| —    | php artisan horizon                      | horizon             |
| 3000 | pnpm dev (Nuxt)                          | web                 |

أو ببساطة **Laravel Sail** للـ API، والـ Nuxt شغال على الجهاز مباشرة.

## 3. هيكل الـ Backend (Laravel)

### 3.1 التقسيم لـ Modules

```
apps/api/app/
├── Modules/
│   ├── — (للمحل Module دائماً شغالة، مش بتظهر كـ Platform) —
│   ├── Identity/      # Tenants, Branches, Users, Roles, Devices, PIN login
│   └── ModuleManager/ # تفعيلها لكل محل + Modules كتالوج الـ
```

|                                     |                                                                       |
|-------------------------------------|-----------------------------------------------------------------------|
| — Billing/                          | # Plans, Subscriptions, Invoices, Payments, Gateways                  |
| — Messaging/                        | # MessageTemplates, MessageLogs, wa.me links                          |
| — Sync/                             | # Push/Pull للأجهزة الأوفلاين                                         |
| — Core (الاشتراكات) —               |                                                                       |
| — Catalog/                          | # Categories, Brands, DeviceModels, Products, Variants, Compatibility |
| — Inventory/                        | # StockLevels, StockMovements, StockLots, SerialItems, Stocktake      |
| — Sales/                            | # Sales, SaleItems, Payments, Returns, Holds (POS)                    |
| — Customers/                        | # Customers, CustomerLedger (الآجل)                                   |
| — Suppliers/                        | # Suppliers, Purchases, SupplierLedger                                |
| — Cash/                             | # Shifts, CashMovements, Expenses                                     |
| — Reports/                          | # Read models + Query classes                                         |
| — Optional (بتنفعّل حسب الاشتراك) — |                                                                       |
| — Repairs/                          | # الصيانة                                                             |
| — Imports/                          | # الاستيراد                                                           |
| — SupplierReturns/                  | # مرجعات الموردين حسب المصدر                                          |
| — UsedDevices/                      | # المستعمل                                                            |
| — Services/                         | # كروت الشحن + تحويلات المحافظ                                        |
| — Installments/                     | # التقسيط                                                             |
| — MultiBranch/                      | # التحويلات بين الفروع + التقارير المجمّعة                            |
| — EInvoicing/                       | # ETA                                                                 |
| — ShopOrders/                       | # الطلبات بين المحلات (شركاء + طلبات)                                 |
| — Support/                          | # Money, Tenancy, Phone, Numbering, Events (Outbox) ...               |

جوه كل Module:

```

Modules/Repairs/
├── module.php          # Manifest: key, نوع, اسم, dependencies, إعدادات, قائمة, صلاحيات
├── RepairsServiceProvider.php # routes/listeners/policies/migrations
├── Contracts/          # تانية مسموح تستخدمها Modules اللي (Interfaces) الواجهة العامة ←
├── Events/             # بيطلعها Module الأحداث العامة اللي ال ← (برصه Public API)
├── Listeners/          # تانية Modules ردود أفعاله على أحداث
├── Models/ Enums/ Actions/ Data/ Policies/
├── Http/ Controllers/ Requests/ Resources/
├── Database/ migrations/ seeders/ factories/
├── routes.php
└── Tests/

```

**الحدود:** أي Module **مايلمشش** Models أو جداول Module تاني. المسموح بس: `Contracts/` و `Events/` بتاعته (وده بيتفرض بـ Pest Arch tests).

## 3.2 قواعد الكود (إجبارية)

- **Controllers رفيعة:** Validation في API Resource ← Action ← Form Request. مفيش Business logic في ال Controller.
- **Action لكل عملية** ( `handle()` واحدة), و Service لما عمليات كثير تشارك نفس ال state.
- كل حاجة Enums: **typed** بدل ال return types, properties, `readonly`, strings.
- **كل عملية بتلمس فلوس أو مخزون جوه** `DB::transaction()`, وال Jobs/Events بتتبع بعد ال commit `(afterCommit)`.

- مفيش `Model::all()` ولا `get()` من غير حد — Pagination دايمآ.
- مفيش `env()` برا ملفات `config`.
- Policies لكل Model — مفيش `if ($user->role == ...)`.
- Migrations قابلة للرجوع (`down()`).
- Pint + Larastan (Level 8) لازم يعدّوا قبل أي merge.

### 3.3 الحزم (Packages)

```
composer require laravel/sanctum laravel/reverb laravel/horizon laravel/pennant \
    spatie/laravel-permission spatie/laravel-query-builder spatie/laravel-activitylog \
    spatie/laravel-medialibrary brick/money propaganistas/laravel-phone \
    dedoc/scramble filament/filament barryvdh/laravel-dompdf # أو spatie/laravel-pdf
composer require --dev pestphp/pest pestphp/pest-plugin-laravel larastan/larastan \
    laravel/pint laravel/telescope
```

| الحزمة                       | ليه                                                        |
|------------------------------|------------------------------------------------------------|
| spatie/laravel-permission    | أدوار وصلاحيات لكل محل (Teams)                             |
| spatie/laravel-query-builder | فلتر وترتيب في الـ APIs بشكل موحد                          |
| spatie/laravel-activitylog   | الـ Audit Log                                              |
| spatie/laravel-medialibrary  | الصور والمرفقات على S3/R2                                  |
| brick/money                  | حسابات الفلوس من غير أخطاء float                           |
| propaganistas/laravel-phone  | توحيد أرقام الموبايل المصرية (wa.me)                       |
| laravel/pennant              | Feature flags حسب الباقة                                   |
| dedoc/scramble               | OpenAPI تلقائي من الكود                                    |
| filament/filament            | لوحة إدارة المنصة (المحلات، الاشتراكات، الموافقات اليدوية) |

### 3.4 نظام الـ Modules (تفعيل لكل محل)

#### الـ Manifest

```
// Modules/Imports/module.php
return new ModuleManifest(
    key: 'imports',
    name: 'الاستيراد',
    tier: ModuleTier::Optional, // Platform | Core | Optional
    dependsOn: ['inventory', 'suppliers'],
    permissions: ['imports.view', 'imports.manage', 'imports.payments'],
    menu: [new MenuItem('imports.index', 'الاستيراد', icon: 'i-lucide-ship', permission:
'imports.view')],
    settings: ['default_cost_allocation' => 'value'],
    seeders: [DefaultImportCostTypesSeeder::class],
)
```

);

## حالة الـ Module لكل محل

| الحالة       | المعنى                     | إليه الي يظهر                                            |
|--------------|----------------------------|----------------------------------------------------------|
| not_entitled | مش في اشتراكه              | مش في القائمة. يظهر بس في صفحة "الـ Modules" بزرار "فعل" |
| trial        | تجربة مؤقتة (مثلاً 14 يوم) | شغال بالكامل + بانر المدة الباقية                        |
| enabled      | في الاشتراك ومتفعل         | شغال بالكامل                                             |
| disabled     | في الاشتراك بس المالك قفله | مخفي، والداتا محفوظة                                     |
| read_only    | الاشتراك خلص أو اتلغى      | يقدر يشوف ويصدّر الداتا القديمة بس                       |

- **Entitlement (من الـ Billing) منفصل عن التفعيل (قرار المالك):** الاشتراك بيقول "مسموح له"، والمالك يقرر "عايز يشوفه ولا لأ".
- **Dependencies:** تفعيل Module بيتأكد إن اللي بيعتمد عليه متفعل، وقفل Module مش مسموح لو في Module متفعل معتمد عليه.
- **أول تفعيل:** بيشغل الـ Seeders بتاعته (مثلاً قوائم الأعطال للصيانة، والقوالب) ويطّلع ModuleEnabled.
- **الإلغاء عمره ما يمسح داتا.** الـ Module بيبقى read\_only، ولو رجع يتفعل كل حاجة ترجع زي ما هي.
- **عند التسجيل:** المالك يختار نوع المحل (إكسسوارات / صيانة / إكسسوارات + صيانة / مستورد / جملة) ← نقترح Modules وباقية مناسبة.

## التطبيق في الكود

```
// Routes
Route::middleware(['auth:sanctum', 'tenant', 'module:imports'])->prefix('imports')->group(...);

// في أي مكان
if (Modules::enabled('repairs')) { ... }

// ModuleGate: Cache محل (Redis) - يسمح مع ModuleEnabled / ModuleDisabled / SubscriptionChanged
final class ModuleGate
{
    public function enabled(string $key, ?Tenant $tenant = null): bool;
    public function state(string $key, ?Tenant $tenant = null): ModuleState;
    /** @return list<string> */
    public function enabledKeys(?Tenant $tenant = null): array;
}
```

- 403 ← module:{key} Middleware module\_not\_enabled لو مش متفعل (والـ Frontend يعرض صفحة "فعل الـ Module").
- الصلاحيات: أدوار المحل بتشوف صلاحيات الـ Modules المتفعله بس.

- ال Listeners بتاعة Module مقبول **مايتشتغلش** للمحل ده (شوف 3.5).
- ال `modules: [{key, state, trial_ends_at}]` بيرجع `GET /api/v1/me` + القائمة الجاهزة — ال `Nuxt` و `Flutter` و `POS` الأوفلاين بينوا القائمة و ال `Routes` منها.
- ال `Sync pull` بيعت داتا ال `Modules` المتفّعة بس.

### Event-Driven Design 3.5

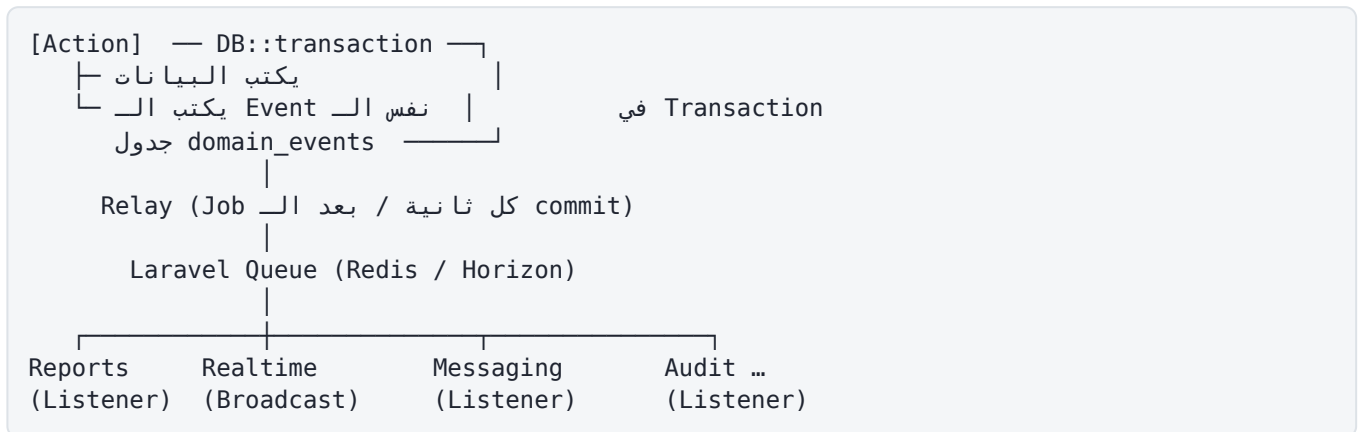
**الفكرة:** كل Module بيعمل شغله وبيعلن "حصل إيه" ك **Domain Event**. ال `Modules` التانية بتسمع وتتصرف. مفيش Module بيكلم التاني عشان "يعمله حاجة" إلا في حالة واحدة (تحت).

### القاعدة الذهبية: Event و Contract؟

| السؤال                                           | الطريقة                                                                                        | مثال                                                                                                                                          |
|--------------------------------------------------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| لو رد الفعل فشل، لازم العملية الأصلية كلها تفشل؟ | <b>Contract</b> (استدعاء مباشر لـ <code>Interface</code> جوه نفس ال <code>Transaction</code> ) | البيع لازم يخضم المخزون، والبيع الآجل لازم يتسجل على حساب العميل                                                                              |
| ممکن يتأخر ثواني أو يتعاد من غير مشكلة؟          | <b>Event</b> (Async في ال <code>Queue</code> )                                                 | التقارير، ال <code>Real time</code> ، الإشعارات، تنبيه النواقص، ال <code>Audit</code> ، عدّادات الاستخدام، اقتراح رسالة <code>WhatsApp</code> |

كده بنكسب الاتنين: **اتساق** الفلوس والمخزون، و**فصل** كامل لكل الباقي.

### Transactional Outbox (عشان ولا Event يضيع)



- ال `Event` بيتسجل في `domain_events` مع البيانات. لو ال `Transaction` فشلت، ال `Event` مش موجود. لو نجحت، مضمون يتبع حتى لو السيرفر وقع.
- ال `Relay` يقرأ الأحداث اللي لسه ماتنشرتش ( `published_at IS NULL` ) بالترتيب وبيعتها لل `Listeners`، ويعلمها.
- **كل Listener Idempotent**: جدول `processed_events (event_id, listener)` Unique — لو الحدث اتبع مرتين، يتنفذ مرة واحدة.
- `Retry` بـ `backoff`، وبعد آخر محاولة يروح **Dead letter** ( `failed_jobs` + تنبيه `Sentry` ) ويتعاد يدوي من `Horizon`.

```
final readonly class SaleCompleted implements DomainEvent
{
    public const NAME = 'sales.sale_completed';
    public const VERSION = 1;

    public function __construct(
        public string $saleId,
        public string $tenantId,
        public string $branchId,
        public ?string $customerId,
        public int $totalPiasters,
        public int $costPiasters,
        public bool $isCredit,
        /** @var list<array{variant_id: string, qty: int, lot_id: string}> */
        public array $items,
        public CarbonImmutable $occurredAt,
    ) {}
}
```

- الأسماء بصيغة الماضي {module}.{something\_happened} ومعناها **Version** عشان نغيّر الشكل من غير ما نكسر ال Listeners القديمة.
- ال Payload فيه اللي ال Listeners محتاجه (IDs + أرقام)، مش Models كاملة.
- كل Event بيحمل `tenant_id` ، وال Listener بيرجع ال Tenant context قبل ما يشتغل.
- **Base listener** بيتأكد إن ال Module بتاعه متفعل للمحل ده، ولو لأ بيتجاهل الحدث:

```
abstract class ModuleListener implements ShouldQueue
{
    abstract protected function module(): string;

    public function handle(DomainEvent $event): void
    {
        if (! Modules::enabled($this->module(), $event->tenantId)) return;
        $this->once($event, fn () => $this->react($event)); // processed_events
    }
}
```

## كتالوج الأحداث (البداية)

| الحدث                             | بيطلع من | مين بيستمع (Async)                                                                                                                  |
|-----------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>sales.sale_completed</code> | Sales    | Realtime, Inventory (إجمالي الوردية), Cash (ملخص يومي), Reports (فحص النواقص ← Messaging , inventory.stock_low ) (اقتراح رسالة شكر) |
| <code>sales.sale_returned</code>  | Sales    | SupplierReturns (لو معيب ← سلة المرتجعات), Realtime, Reports                                                                        |
| <code>sales.sale_voided</code>    | Sales    | Reports, Audit, Realtime (للمالك)                                                                                                   |

| الحدث                                                                  | يطلع من          | مين يسمع (Async)                                                                                               |
|------------------------------------------------------------------------|------------------|----------------------------------------------------------------------------------------------------------------|
| inventory.stock_low                                                    | Inventory        | Realtime, Notifications, Suppliers (اقتراح طلبية)                                                              |
| inventory.item_marked_defective                                        | Inventory        | SupplierReturns                                                                                                |
| purchases.purchase_received                                            | Suppliers        | Reports, Inventory (تنبيه زيادة التكلفة)                                                                       |
| repairs.ticket_created / repairs.status_changed                        | Repairs          | Realtime (الفنيين), Messaging (رسالة مقترحة), Reports                                                          |
| repairs.ticket_delivered                                               | Repairs          | Reports, Cash, Customers (لو آجل — Contract جوه العملية نفسها)                                                 |
| imports.shipment_received                                              | Imports          | Reports, Realtime (للمالك)                                                                                     |
| imports.payment_recorded                                               | Imports          | Reports, Audit                                                                                                 |
| supplier_returns.return_settled                                        | Supplier Returns | Reports                                                                                                        |
| cash.shift_closed                                                      | Cash             | Reports, Realtime (العجز/الزيادة للمالك)                                                                       |
| customers.debt_overdue (Scheduler)                                     | Customers        | Messaging (تذكير مقترح), Notifications                                                                         |
| approvals.requested / approvals.decided                                | Sales/Id entity  | Realtime (تطبيق المالك)                                                                                        |
| modules.module_enabled / module_disabled                               | Module Manager   | Seeders, Cache, Billing (الاستخدام)                                                                            |
| billing.subscription_changed                                           | Billing          | ModuleManager (تحديث الـ Entitlements), Notifications                                                          |
| shop_orders.connection_requested / shop_orders.order_updated (لكل طرف) | ShopOrders       | Realtime, Notifications, Messaging, Suppliers (فاتورة شراء للطلاب) و Sales/Customers (فاتورة بيع + آجل للبائع) |

- الـ **Real time** (قسم 8) = Listener واحد بيدوّل Events مختارة لـ Broadcast على Reverb.
- الـ **Audit log** = Listener ييسجل الأحداث الحساسة.
- الـ **Reports** بتيني **Read models** (زي `daily_sales_summary` , `repair_stats`) من الأحداث بدل ما التقارير تعمل Queries ثقيلة على الجداول الأساسية.
- **مش Event Sourcing**: الجداول هي مصدر الحقيقة، والـ Events للتواصل والتفاعل. (جدول `stock_movements` هو الاستثناء الطبيعي: Ledger ثابت).

- **مستقبلاً:** لو Module اتفصل كخدمة لوحده، ال Relay بيعت الأحداث لـ Broker (RabbitMQ / Redis Streams) بدل ال Queue الداخلي، من غير تغيير في ال Modules.

## 4. Multi-tenancy

قاعدة واحدة + tenant\_id في كل جدول بيانات.

```
// app/Support/Tenancy/BelongsToTenant.php
trait BelongsToTenant
{
    protected static function bootBelongsToTenant(): void
    {
        static::addGlobalScope(new TenantScope);

        static::creating(function (Model $model): void {
            $model->tenant_id ??= app(CurrentTenant::class)->id();
        });
    }

    public function tenant(): BelongsTo
    {
        return $this->belongsTo(Tenant::class);
    }
}
```

- CurrentTenant بيتحدد من المستخدم المسجل (ResolveTenant Middleware)، والفرع من Header X-Branch-Id (ومتأكدين إن المستخدم له صلاحية عليه).
- **خط دفاع ثاني: Row Level Security** Postgres على الجداول الحساسة (SET app.tenant\_id في أول كل request). لو حد نسي ال scope، الداتايز نفسها ترفض.
- **اختبار إجباري** (Pest Arch + Feature): كل Model في Modules/\*/Models لازم يستخدم BelongsToTenant (ماعدا الاستثناءات المعروفة)، وتست إن مستخدم محل A مش بيشف أي بيانات لمحل B.
- ال Queued Jobs بتشيل tenant\_id وبتعمل restore لل context قبل ما تشتغل.
- spatie/laravel-permission بـ teams = true و team\_id = tenant\_id.

## 5. تصميم قاعدة البيانات

### 5.1 قواعد عامة

- ال Primary key: uuid (UUIDv7) لكل الجداول اللي ممكن تتعمل أوفلاين (...sales, repairs, customers, cash)، و bigint للجداول الداخلية (lookup).
- كل جدول: tenant\_id، created\_at، updated\_at، deleted\_at (Soft delete) للكيانات الأساسية.

- الفلوس: bigint بالقرش — عمود اسمه \*\_piasters أو Cast اسمه MoneyCast.
- الكميات: integer (القطع). مفيش كسور في المخزون.
- Index على (tenant\_id, ...) في كل حاجة بيتعمل عليها بحث.
- البحث: Index + pg\_trgm Extension على اسم الصنف والكود والباركود (بحث عربي/إنجليزي جزئي سريع).

## 5.2 الجداول الأساسية (المرحلة الأولى)

|                                                    |                                                                         |                   |
|----------------------------------------------------|-------------------------------------------------------------------------|-------------------|
| tenants                                            | id, name, slug, phone, tax_id, settings(jsonb), created_at              |                   |
| branches                                           | id, tenant_id, name, address, phone, invoice_prefix, is_active          |                   |
| users                                              | id, tenant_id, name, phone, email, password, pin_hash, is_owner         |                   |
| devices                                            | id(uuid), tenant_id, branch_id, name, platform, last_seen_at,           |                   |
| device_code                                        |                                                                         |                   |
| categories                                         | id, tenant_id, parent_id, name, type(enum:                              |                   |
| accessory part device service)                     |                                                                         |                   |
| brands                                             | id, tenant_id, name                                                     |                   |
| device_models                                      | id, tenant_id, brand_id, name, release_year                             | -- كتالوج مشترك + |
| خاص                                                |                                                                         |                   |
| products                                           | id, tenant_id, category_id, brand_id, name, sku, track_serial(bool),    |                   |
| is_active                                          |                                                                         |                   |
| product_variants                                   | id, tenant_id, product_id, name, barcode, quality_grade(enum),          |                   |
|                                                    | price_retail, price_wholesale, price_technician, min_stock              |                   |
| product_compatibility                              | variant_id, device_model_id                                             | -- PK مرّكب       |
| stock_levels                                       | tenant_id, branch_id, variant_id, qty, avg_cost                         | -- PK (branch_id, |
| variant_id)                                        |                                                                         |                   |
| stock_lots                                         | id, tenant_id, branch_id, variant_id, source_type(enum:                 |                   |
| supplier import shipment opening customer_return), |                                                                         |                   |
|                                                    | source_id, source_ref_id, unit_cost, qty_in, qty_remaining, received_at |                   |
| stock_movements                                    | id, tenant_id, branch_id, variant_id, lot_id, type(enum), qty(+/-),     |                   |
|                                                    | unit_cost, ref_type, ref_id, user_id, created_at                        | -- Append-only    |
| serial_items                                       | id, tenant_id, variant_id, branch_id, lot_id, serial, status(enum)      |                   |
| customers                                          | id(uuid), tenant_id, name, phone, phone_e164, notes, credit_limit,      |                   |
| balance                                            |                                                                         |                   |
| customer_ledger                                    | id, tenant_id, customer_id, type, amount, ref_type, ref_id, created_at  |                   |
| suppliers                                          | id, tenant_id, name, phone, type(enum: local shop), balance             |                   |
| supplier_ledger                                    | id, tenant_id, supplier_id, type, amount, ref_type, ref_id              |                   |
| purchases / purchase_items                         |                                                                         |                   |
| sales                                              | id(uuid), tenant_id, branch_id, device_id, shift_id, number,            |                   |
| customer_id,                                       |                                                                         |                   |
|                                                    | status, subtotal, discount, total, paid, due, cost_total, completed_at  |                   |
| sale_items                                         | id, sale_id, variant_id, lot_id, serial_item_id, qty, unit_price,       |                   |
| discount, unit_cost                                |                                                                         |                   |
| sale_payments                                      | id, sale_id, method(enum), amount, reference                            |                   |
| shifts                                             | id(uuid), tenant_id, branch_id, user_id, opened_at, closed_at,          |                   |
| opening_cash, expected(jsonb), actual(jsonb)       |                                                                         |                   |
| cash_movements / expenses / expense_categories     |                                                                         |                   |
| activity_log                                       | (spatie)                                                                |                   |

## 5.3 جداول الصيانة

|                  |                                                                   |                       |
|------------------|-------------------------------------------------------------------|-----------------------|
| fault_categories | id, tenant_id, name, sort                                         | -- شاشة، بوردة، سوكيت |
| شحن...           |                                                                   |                       |
| fault_types      | id, tenant_id, category_id, name, default_labor_price, is_active, |                       |
| sort             |                                                                   |                       |

```
intake_checklist_items id, tenant_id, label, sort -- ... بيفتح؟ بيشحن؟
repair_tickets id(uuid), tenant_id, branch_id, number, customer_id,
device_model_id, imei, color,
passcode_encrypted, accessories(jsonb), condition(jsonb),
intake_checks(jsonb),
status(enum), technician_id, received_at, received_by, expected_at,
delivered_at, labor_price, deposit, total, warranty_days, notes
repair_ticket_faults ticket_id, fault_type_id, kind(enum: reported|diagnosed), note
repair_status_logs id, ticket_id, from_status, to_status, user_id, note, created_at
repair_parts id, ticket_id, variant_id, lot_id, qty, unit_price, unit_cost
```

الأعطال الافتراضية (الجدول الي في PLAN.md) بتعمل **Seed** لكل محل جديد وقت التسجيل.

## 5.4 جداول الاستيراد ومرتجات الموردين

```
import_contacts id, tenant_id, type(enum: factory|agent|shipping|clearance), name,
country, phone, wechat, notes
import_shipments id, tenant_id, contact_id, number, status(enum), ordered_at,
expected_at, arrived_at,
cost_allocation(enum: value|qty|weight), notes
import_shipment_items id, shipment_id, variant_id, qty_ordered, qty_received,
qty_damaged, unit_price, landed_unit_cost, weight
import_costs id, shipment_id, type(enum:
shipping|customs|clearance|transport|commission|other), contact_id, amount
import_payments id, tenant_id, contact_id, shipment_id(nullable), amount, method,
paid_at,
reference, receiver, original_amount_note, proof(media)
import_claims id, shipment_id, type(enum: shortage|damaged), amount, status

supplier_returns id, tenant_id, source_type, source_id, number, status(enum:
pending|sent|accepted|rejected), sent_at, settled_at
supplier_return_items id, return_id, variant_id, lot_id, serial_item_id, qty, unit_cost,
reason(enum), outcome(enum: credit|replace|refund|rejected)
```

- حساب كل جهة استيراد =  $\Sigma$  (تكلفة الشحنات + المصاريف المنسوبة ليها) -  $\Sigma$  الدفعات -  $\Sigma$  المطالبات المقبولة. بيتحسب في **View** أو Query class، مش عمود بيتعدل بإيد.

## 5.5 جداول الرسائل والـ Billing

```
message_templates id, tenant_id, key, context(enum:
repair|sale|customer|supplier|import|general), name, body, is_active
message_logs id, tenant_id, template_id, user_id, recipient_phone, context_type,
context_id, rendered_body, opened_at
public_links id(token), tenant_id, linkable_type, linkable_id, expires_at --
لينكات الفواتير والإيصالات

plans, plan_features, subscriptions, subscription_items, billing_invoices,
billing_invoice_lines,
billing_payments, payment_methods, coupons, credit_ledger, usage_counters, billing_events
```

(التفاصيل في PLAN.md قسم 3.9 — الجداول دي **مش** عليها tenant\_id scope الأوتوماتيكي لأنها بتاعة المنصة.)

## 5.5.1 جداول الطلبات بين المحلات

```

tenants.code          بيتعمل تلقائي - (unique) كود المحل القصير
shop_connections      id, requester_tenant_id, addressee_tenant_id,
status(pending|accepted|declined), requested_by, responded_at
shop_orders           id, number (sequence → S0-000123), buyer_tenant_id,
seller_tenant_id, type(goods|repair), status,
                      needed_by, notes, total, placed_by, accepted_at, ready_at,
delivered_at, completed_at, closed_at
shop_order_items      id, shop_order_id, buyer_tenant_id, seller_tenant_id, description,
quantity, unit_price, device_model, imei, note
shop_order_activities id, shop_order_id, buyer_tenant_id, seller_tenant_id, from_status,
to_status, actor_party, actor_user_id, note, created_at

```

- الصفوف دي بتاعة **محلين في نفس الوقت**، فبتستخدم SharedBetweenTenants بدل BelongsToTenant : المحل يشوف الصف لو هو واحد من الطرفين، ومن غير محل مفيش نتايج.
- الوصول لبيانات المحل الثاني (الاسم، الكود، الموبايل) عن طريق Contract Identity\Contracts\ShopDirectory (بحث بالكود بالظبط بس).
- كل خطوة بتطّلع حدث shop\_orders.order\_updated **مرة لكل محل** عشان الـ Listeners (إشعارات، Real time، وبعدين فاتورة الشراء/البيع) تشتغل في context كل محل.

## 5.6 جداول الـ Modules و الـ Events

```

modules              key(PK), name, tier(enum: platform|core|optional), depends_on(jsonb),
is_public, sort
plan_modules         plan_id, module_key -- الباقه فيها أنهى --
Modules
tenant_modules       tenant_id, module_key, entitled(bool), state(enum:
trial|enabled|disabled|read_only),
source(enum: plan|addon|trial|manual), trial_ends_at, enabled_at,
disabled_at, settings(jsonb)
domain_events        id(uuid), tenant_id, name, version, aggregate_type, aggregate_id,
payload(jsonb),
occurred_at, published_at, attempts -- الـ Outbox
(Index على published_at IS NULL)
processed_events     event_id, listener, processed_at -- مرّكب PK
(Idempotency)

```

- domain\_events بيتعمله Partition شهري أو أرشفة بعد 90 يوم عشان حجمه.

## 6. محرّك المخزون (أهم جزء في الـ Business logic)

**القاعدة:** stock\_movements هو مصدر الحقيقة (Append-only، مفيش update ولا delete). stock\_levels مجرد أرصدة بتتحدّث في نفس الـ Transaction. stockLots.qty\_remaining g

```

final class CompleteSaleAction
{
    public function __construct(

```

```

        private readonly StockLedger $stock,           // Inventory\Contracts – لازم ينجح
مع البيعة
        private readonly CustomerAccounts $accounts,   // Customers\Contracts – لازم ينجح
مع البيعة
        private readonly EventRecorder $events,       // Support\Events – بيكتب في الـ
Outbox
    ) {}

    public function handle(SaleData $data, User $cashier): Sale
    {
        return DB::transaction(function () use ($data, $cashier): Sale {
            // Idempotency: اتسجل قبل كده (إعادة مزامنة) رجّعه زي ما هو UUID لو الـ
            if ($existing = Sale::find($data->id)) {
                return $existing;
            }

            $sale = Sale::create([...]);

            foreach ($data->items as $item) {
                // lockForUpdate على stock_levels + الـ Lots استهلاك الـ FIFO
                $consumed = $this->stock->consume(
                    branchId: $data->branchId,
                    variantId: $item->variantId,
                    qty: $item->qty,
                    reference: StockReference::sale($sale->id),
                    serialId: $item->serialItemId,
                );
                $sale->items()->create([...$item->toArray(), 'lot_id' => $consumed->lotId,
'unit_cost' => $consumed->unitCost]);
            }

            $sale->payments()->createMany($data->payments);

            if ($sale->due > 0) {
                $this->accounts->charge($sale->customer_id, $sale->due, reference: "sale:
{$sale->id}");
            }

            // بتسمع للحدث ده Listeners (...نواقص، رسائل، Real time، تقارير) الباقي كله
            $this->events->record(SaleCompleted::fromSale($sale));

            return $sale;
        });
    }
}

```

- **التكلفة:** متوسط مرجّح (avg\_cost) للتقارير، وكل بيعة بتأخذ unit\_cost من الـ Lot المستهلك (FIFO) — ده اللي بيخّلي المرتجع يعرف مصدره والمكسب دقيق.
- البيع بالسالب مسموح (خصوصاً في الأوفلاين) بس بيطلع تنبيه تسوية.
- **الاستيراد:** عند استلام الشحنة ← حساب landed\_unit\_cost لكل صنف ← إنشاء stock\_lots بمصدر import\_shipment.
- **المرتجع من عميل:** يرجع لـ Lot بتاعه (سليم) أو يروح supplier\_returns بمصدر الـ Lot (معيّب).
- **التحويل بين الفروع:** Movement خروج من فرع + دخول لفرع بنفس الـ Lot source (عشان المصدر مبيضيعش).

## 7. الكاشير الأوفلاين والمزامنة

### 7.1 على الجهاز (Nuxt PWA)

- Service Worker يكاش الـ App shell فيفتح من غير نت. @vite-pwa/nuxt
- **Dexie (IndexedDB)** فيه نسخة من: الأصناف والأسعار والباركود، المخزون الحالي للفرع، العملاء، الإعدادات، القوالب.
- **Outbox**: كل عملية (بيعة، مرتجع، عميل جديد، حركة خزنة، تذكرة) بتسجل محلياً بـ UUIDv7 وتتحت في طابور.
- ترقيم الفواتير محلياً: {branch\_prefix}-{device\_code}-{seq} — مفيش تعارض بين الأجهزة.
- حالة الاشتراك متكاشة على الجهاز مع مهلة 7 أيام أوفلاين.

### 7.2 API

```
POST /api/v1/sync/push { device_id, operations: [ {id, type, payload, created_at}, ... ] }
← { results: [ {id, status: applied|duplicate|rejected, error?} ] }

GET /api/v1/sync/pull?since=<cursor>&entities=products,stock,customers,templates
← { changes: {...}, next_cursor }
```

- الـ Push بيتنقذ بالترتيب وكل عملية Idempotent على الـ id.
- الـ Pull بـ cursor على updated\_at + id (Delta sync)، وأول مرة Full snapshot.
- التعارضات: السيرفر هو المرجع للأسعار والأصناف، والجهاز هو المرجع لعملياته (البيعة اللي حصلت حصلت).
- فشل عملية (مثلاً صنف اتمسح) ← تتعلم rejected وتظهر للمدير يحلها يدوي — **عمرها ما تتمسح بصمت**.

### 7.3 الترتيب في التنفيذ

1. **MVP**: الكاشير أونلاين بس يكتب في Dexie الأول (Local-first UI) وبيعت فوراً.
2. **Beta**: تفعيل الأوفلاين الكامل + الـ Outbox + شاشة "عمليات مستتية مزامنة".

## 8. الـ Real time (Reverb)

| مين بيسمع                 | Events                                                                   | Channel                           |
|---------------------------|--------------------------------------------------------------------------|-----------------------------------|
| لوحة الفرع، الكاشير       | SaleCompleted , StockLow , RepairStatusChanged , ShiftClosed             | private-tenant.<br>{t}.branch.{b} |
| تطبيق المالك، لوحة المالك | ApprovalRequested (خضم كبير/مرتجع),<br>DailySummaryReady , ملخص المبيعات | private-tenant.<br>{t}.owner      |

| مين يسمع     | Events                                    | Channel                        |
|--------------|-------------------------------------------|--------------------------------|
| شاشة الفنيين | RepairTicketCreated , RepairStatusChanged | private-tenant.<br>{t}.repairs |
| المستخدم     | إشعارات شخصية                             | private-user.{u}               |

- Authorization لل Channels في routes/channels.php يتأكد من ال tenant والفرع.
- Flutter . pusher-js + laravel-echo :Nuxt: dart\_pusher\_channels (بروتوكول Pusher الي Reverb يبدعه).
- ال Broadcasting نفسه **Listener** على ال Domain Events (بعد ما تنشر من ال Outbox)، فمحدث يستلم حدث لعملية أترجعت، وال Modules مش عارفة حاجة عن Reverb.
- ال Channels الخاصة بـ Module مقفول مايتفتحش (ال Authorization يتأكد من Modules::enabled).
- Payload صغير (IDs + أرقام أساسية)، والعمل يعمل refetch لو محتاج تفاصيل.

## 9. الطباعة والباركود

- **المرحلة 1:** طباعة من المتصفح بـ CSS مخصص لـ 80mm/58mm ( @page { size: 80mm auto } ) — بتشغل مع أي طابعة حرارية متعرفة على الويندوز.
- **المرحلة 2: QZ Tray** (أو Print agent صغير) لطباعة ESC/POS مباشرة من غير نافذة الطباعة + فتح درج الكاش.
- لبيلات الباركود: JsBarcode + قالب طباعة لمقاسات الليبل الشائعة (مثلاً 25mm×38).
- قراءة الباركود: القارئ USB ييشغل كـ Listener ← Keyboard في ال POS يفرق بين الكتابة السريعة (سكانر) والكتابة العادية. + مسح بالكاميرا في الموبايل ( @zxing/browser ).
- الفاتورة A4 و PDF ← PDF: Blade template أو DomPDF أو Browsershot لدعم العربي الأحسن).

## 10. قوالب WhatsApp

```
final class RenderMessageAction
{
    public function handle(MessageTemplate $template, Model $context, User $user):
    RenderedMessage
    {
        $vars = $this->variablesFor($context); // {رقم_التذكرة}, {اسم_العميل},
        {المبلغ}...
        $body = strstr($template->body, $vars);
        $phone = phone($this->recipientPhone($context), 'EG')->formatE164(); // +2010...

        $log = MessageLog::create([...]); // سجل "اتفتحت"
        return new RenderedMessage(
            body: $body,
            url: 'https://wa.me/'.ltrim($phone, '+').'?text='.rawurlencode($body),
        );
    }
}
```

```

    }
}
);

```

- ال Frontend يعرض الرسالة للتعديل، وبعددين `window.open(url)`.
- لينك الفاتورة/الإيصال: `https://app.domain/r/{token}` (جدول `public_links` , token عشوائي 32 حرف، ومفيهوش بيانات حساسة).
- القوالب الافتراضية بتتعمل Seed لكل محل جديد.

## 11. ال Billing (تقنياً)

```

interface PaymentGateway
{
    public function createCheckout(BillingInvoice $invoice, PaymentMethodType $method):
CheckoutSession;
    public function chargeSavedMethod(BillingInvoice $invoice, PaymentMethod $method):
ChargeResult;
    public function verifyWebhook(Request $request): WebhookEvent; // HMAC
}
// Drivers: PaymobGateway, FawryGateway, ManualGateway (InstaPay/مندوب)

```

- Webhooks: `POST /webhooks/{gateway}` ← تحقق HMAC ← `billing_events` ب Unique على `gateway_ref` (Idempotent) ← Job يطبق الدفع.
- **Scheduler يومي:** `billing:generate-invoices` , `billing:charge-due` , `billing:transition-states` (restricted) ← `past_due` ← `suspended` ← `billing:send-reminders`.
- الاشتراك = باقة ( `plan_modules` ) + Modules إضافية ( `subscription_items` نوعها `module` ). أي تغيير ← `billing.subscription_changed` ← `ModuleManager` يحدّث `tenant_modules.entitled`.
- تجربة Module لوحده (مثلاً 14 يوم استيراد) من صفحة ال Modules، وبعدها يا يشترك يا يبقى `read_only`.
- Middleware `module:{key}` لل Modules، و `Pennantg` لل features الصغيرة جوه ال Module، و `PlanLimits` للحدود (مستخدمين، فروع، أصناف).
- وضع RESTRICTED: القراءة والبيع مسموحين، الإنشاء في الكيانات المحددة ممنوع (403 برسالة واضحة + بانر).
- لوحة Filament: المحلات، الاشتراكات، الموافقة على تحويلات InstaPay (صورة الإيصال)، الكوبونات، المندوبين.

## 12. ال API Conventions

- Prefix: `/api/v1/...` — `Route::apiResource` لل CRUD.
- الرد دائماً API Resource: `{ "data": {...}, "meta": {...} }` json

- شكل الأخطاء موحّد (Exception handler): { "message": "الرصيد غير كافٍ", "code": "insufficient\_credit", "errors": { "field": ["..."] } }
- الأكواد: 201 إنشاء، 403 422 Validation، 404 صلاحيات/باقة، 409 تعارض، 429 Rate limit.
- Pagination: Cursor للقوائم الكبيرة (المبيعات، الحركات)، و Page للإعدادات.
- فلترة وترتيب: ?filter[status]=ready&sort=-received\_at&include=customer (spatie/query-builder).
- Rate limiting على Login و PIN و Sync و Webhooks.
- Accept-Language: ar — رسائل ال Validation بالعربي.

## 13. ال (Nuxt 4) Frontend

```
pnpm dlx nuxi@latest init apps/web
pnpm add @nuxt/ui @pinia/nuxt @nuxtjs/i18n @vite-pwa/nuxt @vueuse/nuxt dexie \
  laravel-echo pusher-js jsbarcode @zxing/browser echarts vue-echarts zod
pnpm add -D vitest @vue/test-utils @playwright/test openapi-typescript
```

```
apps/web/app/
├── pages/
│   ├── pos/ # اختصارات (Fullscreen، مستقل layout) الكاشير
│   ├── repairs/ # التذاكر + استلام جهاز
│   ├── inventory/ products/ purchases/ suppliers/ customers/
│   ├── imports/ supplier-returns/
│   └── cash/ reports/ settings/ billing/
├── layouts/ default.vue (Sidebar RTL) · pos.vue
├── composables/ useApi.ts · useAuth.ts · useBranch.ts · useModules.ts · useScanner.ts ·
  useWhatsApp.ts · useEcho.ts
├── middleware/ module.ts ("Module مش متفعل ← صفحة "فعل ال Module ل Route يمنع دخول)
├── modules-registry.ts # key ← routes + ال widgets عناصر القائمة
├── stores/ (Pinia) auth · cart · settings · sync
├── offline/ db.ts (Dexie schema) · outbox.ts · sync.ts
└── utils/ money.ts (قرش جنيه) · phone.ts · print.ts
```

- **RTL أولًا:** dir="rtl" + خط **Cairo** أو IBM Plex Sans Arabic، والأرقام إنجليزي (أوضح في الكاشير) مع إعداد لتحويلها.
- Types من ال API: openapi-typescript على ال spec اللي Scramble بيطلع ← packages/api-types.
- Auth: Sanctum SPA cookies، واللوحة، وال POS بـ Device token + دخول الكاشير بـ PIN.
- أداء الكاشير: البحث في Dexie (مش في السيرفر)، هدف إضافة صف > 100ms.
- **ال Modules في الواجهة:** القائمة وال Dashboard widgets والأزرار (زي "تحويل لتذكيرة صيانة" في الكاشير) بتبني من modules اللي راجعة من ./me. كل صفحة Module عليها { module: 'imports' } definePageMeta. ويفضل Layers في Nuxt لكل Module كبير (layers/repairs، layers/imports) عشان الكود يفضل منفصل.
- صفحة **"ال Modules"** في الإعدادات: كل ال Modules بوصف وسعر، والمتفعل منها، ووزار تجربة/اشتراك/إخفاء.

## 14. تطبيق المالك (Flutter)

- `firebase_messaging` , `dart_pusher_channels` , `json_serializable` + `freezed` , `go_router` , `dio` , `flutter_riverpod` (Push) , `mobile_scanner` , `fl_chart` (الجرد بالكاميرا).
- الشاشات: الملخص اللحظي، الموافقات، التذاكر، النواقص، الآجل، الاستيراد (الأرصدة والشحنات)، التقارير.
- Architecture: Feature-first ( `features/dashboard` , `features/approvals` ... ) + Repository لكل Feature.
- يتبني بعد ما ال API يستقر (المرحلة P1) — مش من أول يوم.

## 15. الأمان

- تشفير على مستوى الحقل ( `cast encrypted` ): الباسورد/النمط، الرقم القومي. صور البطائق في Bucket خاص بـ Signed URLs قصيرة العمر.
- 2FA للمالك (Fortify), PIN للكاشر مع قفل بعد 5 محاولات.
- إدارة الأجهزة: المالك يشوف الأجهزة المتصلة ويلغي أي Token.
- Audit Log لكل العمليات الحساسة (إلغاء، مرتجع، تعديل سعر، خصم فوق الحد، فتح الدرج، رؤية بيانات مشفرة).
- HTTPS, CSP headers فقط, Cookies `Secure` / `HttpOnly` / `SameSite=Lax`.
- `pnpm audit` و `composer audit` في ال CI.

## 16. الاختبارات و ال CI

| الطبقة      | الأداة    | يغطي إيه                                                                                                                                                                                             |
|-------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arch        | Pest Arch | Controllers مفياش DB, كل Model عليه <code>tenant trait</code> , مفيش ( <code>env</code> ) , مفيش <b>Module</b> بيستخدم <b>Models Module</b> تاني ( <code>Events</code> و <code>Contracts</code> بس ) |
| Modules     | Pest      | Route لـ Module مقفول ← 403, Listener مايبشتغلش لمحل الـ Module مقفول عنده، الـ <code>Dependencies</code> , الإلغاء مايبمسحش داتا                                                                    |
| Events      | Pest      | كل Action بيسجل الحدث الصح في الـ <code>assertRecorded : Events</code> (Outbox), الـ <code>Listeners Idempotent</code> (نفس الحدث مرتين ← أثر واحد), Rollback ← مفيش حدث                             |
| Unit        | Pest      | Money, توزيع تكلفة الاستيراد, Proration, FIFO, تجهيز رقم الموبايل                                                                                                                                    |
| Feature     | Pest      | كل Endpoint: نجاح + Validation + صلاحيات + عزل المحلات                                                                                                                                               |
| Concurrence | Pest      | بيعتين في نفس اللحظة لنفس الصنف ← المخزون صح                                                                                                                                                         |

| الطبقة        | الأداة     | يغطي إيه                                                                                                      |
|---------------|------------|---------------------------------------------------------------------------------------------------------------|
| Frontend unit | Vitest     | Outbox, الحسابات, الـ Cart                                                                                    |
| E2E           | Playwright | رحلة: فتح وردية ← بيع ← مرتجع ← قفل وردية. واستلام جهاز صيانة ← تسليم. ووضع الأوفلاين (قطع النت وتكملة البيع) |

GitHub Actions (على كل PR):

```
api: composer install → pint --test → larastan (level 8) → pest --parallel (Postgres service)
web: pnpm install → nuxi typecheck → eslint → vitest → nuxi build
e2e: (على main) docker compose up → playwright
```

## 17. النشر (DevOps)

| البيئة     | المكان                                                  |
|------------|---------------------------------------------------------|
| Local      | Docker compose / Sail                                   |
| Staging    | VPS صغير (نفس إعداد الـ Production) — كل merge على main |
| Production | 4 vCPU / 8GB VPS (Hetzner/DigitalOcean) في البداية      |

- **Docker images**: api (FrankenPHP أو php-fpm + nginx), worker (Horizon), reverb, scheduler, web (Nuxt) — والإدارة بـ Coolify أو Laravel Forge.
- Postgres: على VPS منفصل أو Backup Managed, يومي (pgBackRest أو pg\_dump) لـ Cloudflare R2 + اختبار استرجاع شهري.
- Reverb ورا Nginx على wss://ws.domain.
- الملفات على Cloudflare R2, والـ DNS و الـ CDN على Cloudflare.
- مراقبة: Telescope, Uptime monitor, Horizon dashboard, Sentry (Laravel + Nuxt + Flutter) على الـ Staging بس.
- Migrations بـ zero-downtime: إضافة عمود ← نشر ← Backfill ← إزالة القديم في إصدار بعده.

## 18. خطة التنفيذ التفصيلية

### الأسبوع 0 — التجهيز (أول يوم ويومين)

```
# 1) الـ Repo
mkdir -p apps packages infra docs

# 2) Laravel
```

```
composer create-project laravel/laravel apps/api
cd apps/api
php artisan install:api # Sanctum + routes/api.php
php artisan install:broadcasting # Reverb + Echo config
composer require laravel/horizon spatie/laravel-permission spatie/laravel-query-builder \
    spatie/laravel-activitylog brick/money propagandistas/laravel-phone dedoc/scramble
laravel/pennant
composer require --dev pestphp/pest pestphp/pest-plugin-laravel larastan/larastan
laravel/pint
php artisan horizon:install
./vendor/bin/pest --init

# 3) Nuxt
cd ../ && pnpm dlx nuxi@latest init web && cd web
pnpm add @nuxt/ui @pinia/nuxt @nuxtjs/i18n @vite-pwa/nuxt @vueuse/nuxt dexie laravel-echo
pusher-js

# 4) Docker compose (pgsql, redis, mailpit, minio) + CI workflow + .editorconfig
```

- ☒ Postgres بدل SQLite في .env , APP\_LOCALE=ar , APP\_TIMEZONE=Africa/Cairo

- ☒ Pest + Larastan + Pint شغالين في ال CI على أول PR.

## الأسبوع 1-3 — الأساس (Identity + Catalog)

| الأسبوع | المهام                                                                                                                                                                                                                                                                                       |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | Tenancy (RLS + middleware + scope + trait + تست العزل)، هيكل ال <b>Modules (Manifest + ServiceProvider loader + ال ModuleGate + middleware)</b> ، ال <b>Outbox (EventRecorder + Relay + ModuleListener + processed_events)</b> ، تسجيل محل جديد باختيار نوع المحل، Login، الفروع، Money cast |
| 2       | المستخدمين والأدوار والصلاحيات، الأجهزة و Layout، Audit log، PIN، Auth ال Nuxt RTL + اختيار الفرع                                                                                                                                                                                            |
| 3       | التصنيفات، الماركات، الموديلات، الأصناف والمتغيرات، التوافق، الباركود، بحث pg_trgm، استيراد أصناف من Excel                                                                                                                                                                                   |

## الأسبوع 4-8 — MVP البيع

| الأسبوع | المهام                                                                                          |
|---------|-------------------------------------------------------------------------------------------------|
| 4       | محرك المخزون (movements, levels, lots, FIFO, avg cost) + تستات ال Concurrency، الرصيد الافتتاحي |
| 5       | المشتريات والموردين وحساباتهم، مرتجع المشتريات                                                  |
| 6       | ال POS: شاشة البيع، البحث من Dexie، السلة، الخصومات، طرق الدفع، الفواتير المعلقة، الطباعة 80mm  |
| 7       | العملاء والآجل وكشف الحساب، المرتجعات من العملاء، الورديات والخزنة والمصروفات                   |
| 8       | التقارير الأساسية + لوحة المالك + (SaleCompleted, StockLow) Reverb + قوالب WhatsApp (الأساس)    |

## الأسبوع 9-11 — الصيانة

- قوائم الأعطال + Checklist الاستلام + Seed افتراضي.
- استلام الجهاز، الإيصال ب QR، الحالات وال Timeline، القطع المسحوبة، العمولة، الضمان.

- قوالب رسائل الصيانة + شاشة الفنيين Real time + تقارير الأعطال.

## الأسبوع 12-14 — الاستيراد ومرتجعات الموردين

- جهات الاستيراد، الشحنات، المصاريف وتوزيع التكلفة، الاستلام والنواقص.
- الدفعات وكشف الحساب.
- سلة المرتجعات، الفرز حسب المصدر، إذن المرتجع، الحالات والتأثير على الحسابات.

## الأسبوع 15-17 — ال Billing

- الباقيات والحدود و Pennant، التجربة، Paymob + Fawry + Manual، ال Webhooks، الفواتير، ال Scheduler والتذكيرات، ال Filament للإدارة.

## الأسبوع 18-21 — Beta

- الأوفلاين الكامل (Outbox + Sync push/pull)، تحسين السرعة، 10-20 محل تجريبي، إصلاحات، Sentry، Backups، ال Production.

## بعد الإطلاق

- تطبيق المالك (Flutter)، المستعمل، الشحن والتحويلات، الفروع والتحويلات، ETA، QZ Tray، المتجر الأولين.

## 19. Definition of Done (لكل Feature)

- Migration قابلة للرجوع + Indexes.
- Model عليه BelongsToTenant + Policy + Casts.
- ال Feature جوه Module واحد، وال Routes عليها module:{key} لو Optional، والصلاحيات في ال Manifest.
- أي تأثير على Module ثاني عن طريق Contract (لو لازم يبقى في نفس العملية) أو Event (غير كده) — ومتسجل في كتالوج الأحداث.
- Form Request + Action + API Resource.
- تستات Feature (نجاح، Validation، صلاحيات، عزل المحلات).
- الشاشة في Nuxt بالعربي RTL وشغالة على شاشة الموبايل.
- Audit log للعمليات الحساسة.
- Pint + Larastan + Pest + Typecheck كلهم أخضر.
- اتجرب على ال Staging.

## 20. أول 5 مهام تبدأ بيها النهارده

1. إنشاء الـ Monorepo + Laravel + Nuxt + Docker compose (Postgres/Redis).
2. إعداد الـ CI (Pint + Larastan + Pest) من أول commit.
3. Tenant + Branch + User trait الـ Tenancy + تست العزل + هيكل الـ Modules والـ Outbox (قبل أي Feature، عشان كل حاجة بعد كده تتبني عليهم).
4. تسجيل محل جديد (Onboarding API) باختيار نوع المحل ← Modules مقترحة، مع Seed للأعطال والقوالب والأدوار.
5. Nuxt الـ Layout بالعربي Login + RTL + اختيار الفرع.